

DOATokenV2 – Manual Técnico y Operativo

Índice General

- #1–#47: Scripts de validación, verificación y monitoreo
- #48–#60: Scripts de chequeo y validación
- #61–#66: Scripts de auditoría y análisis de eventos
- #67–#75: Scripts de diagnóstico y pruebas
- #76–#77: Utilidades y limpieza
- #78–#84: Estructura de carpetas

Scripts de Validación y Monitoreo (#1–#47)

(Ya documentados en cuadros individuales: `liquidityMonitor.js`, `validateLiquidity.js`, `verify.js`, `visualizeEvents.js`, etc.)

Scripts de Chequeo y Validación (#48–#60)

- #48 `check-deploy.js`
- #49 `check-owner.js`
- #50 `check-wallet.js`
- #51 `check-gas.js`
- #52 `checkBalances.js`
- #53 `checkBalancesAdmin.js`
- #54 `checkBalancesOwner.js`
- #55 `checkAllBalances.js`
- #56 `checkPair.js`
- #57 `checkPools.js`
- #58 `checkPoolsAll.js`
- #59 `checkReserves.js`
- #60 `checkTotalSupply.js`

(Cada uno ya en cuadro independiente con descripción, comando y variables `.env`)

Scripts de Auditoría y Eventos (#61–#66)

- #61 `analyzeEvents.js`
- #62 `auditToken.js`
- #63 `exportDeploymentsToCSV.cjs`
- #64 `deploymentsReport.cjs`
- #65 `deploymentsFullReport.cjs`

- #66 getLastDeployment.cjs

Scripts de Diagnóstico y Pruebas (#67–#75)

- #67 diagnose.js
- #68 testBalance.js
- #69 testRpc.js
- #70 testEthRpc.js
- #71 testRpcCombined.js
- #72 testSwaps.cjs
- #73 quickCheck.js
- #74 getPairAddress.js
- #75 check-nonce.js

Utilidades y Limpieza (#76–#77)

- #76 cleanEntityTags.js
- #77 .gitignore

Estructura de Carpetas (#78–#84)

- #78 /contracts → contratos inteligentes
- #79 /scripts → scripts operativos
- #80 /logs → registros auditables
- #81 /docs → documentación técnica y legal
- #82 /dashboard → panel visual y UI
- #83 /config → configuración de red y entorno
- #84 /tests → pruebas y validaciones
- #85 .env — variables de entorno críticas
- #85 .env — variables de entorno críticas
- #86 .last_deploy — último despliegue (hash/fecha/contrato)
- #87 addresses.json — direcciones de contratos y wallets
- #88 balances.csv — exportación de balances
- #89 burn-log.json — registro auditable de quemas
- #90 deployments.json / .csv / .old — historial de despliegues
- #91 lp-distribution.json — configuración de distribución de LP
- #92 pairs.json — configuración de pares DOA/base
- #93 ownership-signature.txt — evidencia criptográfica de propiedad
- #94 README.md — documentación base del proyecto
- #95 SECURITY.md — políticas y buenas prácticas de seguridad
- #96 /audit — auditorías externas de contratos y liquidez
- #97 /automation — scripts automatizados operativos
- #98 /compliance — AML/KYC y certificación legal
- #99 /cex-application — aplicación y anexos para CEX
- #100 /FundadorUnico — documentación de fundador único

- **#101** /reports — reportes adicionales y métricas
- **#102** /signer — firmas digitales y materiales de validación
- **#103** /tokenlist y /tokenlists — listas oficiales de tokens
- **#104** /data — datasets auxiliares
- **#105** /docs.old — documentación histórica
- **#106** BCO...png — branding/logo del proyecto
- **#107** liquidityAccumulatedChart.png — gráfico acumulado de liquidez
- **#108** liquidityChart.png — gráfico de liquidez en tiempo real

1 Script: liquidityMonitor.js

Descripción: Monitorea reservas y slippage.

Comando: node scripts/liquidityMonitor.js

Variables .env: PAIR_ADDRESS, ALERT_THRESHOLD

2 Script: validateLiquidity.js

Descripción: Valida estado de liquidez.

Comando: node scripts/validateLiquidity.js

Variables .env: PAIR_ADDRESS

3 Script: validateReserves.js

Descripción: Verifica reservas actuales.

Comando: node scripts/validateReserves.js

Variables .env: PAIR_ADDRESS

4 Script: verify.js

Descripción: Verifica contrato en explorer.

Comando: node scripts/verify.js

Variables .env: CONTRACT_ADDRESS, CONSTRUCTOR_ARGS

5 Script: verifyOwnershipOwner.js

Descripción: Verifica Owner actual.

Comando: node scripts/verifyOwnershipOwner.js

Variables .env: CONTRACT_ADDRESS

6 Script: verifyPools.js

Descripción: Verifica pools desplegadas.

Comando: node scripts/verifyPools.js

Variables .env: POOL_CONFIG.json

7 Script: visualizeEvents.js

Descripción: Muestra eventos Transfer/Mint/Burn.

Comando: node scripts/visualizeEvents.js

Variables .env: CONTRACT_ADDRESS

8 Script: governance-dashboard.ps1

Descripción: Ejecuta un panel interactivo de gobernanza con métricas de liquidez y supply.

Comando: powershell ./scripts/governance-dashboard.ps1

Variables .env: DEPLOYMENTS.json, balances.csv

9 Script: governance-dashboard-interactive.ps1

Descripción: Versión interactiva del dashboard con menús y selección de métricas.

Comando: powershell ./scripts/governance-dashboard-interactive.ps1

Variables .env: DEPLOYMENTS.json, balances.csv

10 Script: governance-annual.ps1

Descripción: Genera reporte anual de gobernanza con métricas acumuladas.

Comando: powershell ./scripts/governance-annual.ps1

Variables .env: balances.csv, burn-log.json

11 Script: governance-quarterly.ps1

Descripción: Reporte trimestral de gobernanza y liquidez.

Comando: powershell ./scripts/governance-quarterly.ps1

Variables .env: balances.csv, liquidity-events.log

12 Script: governance-executive.ps1

Descripción: Reporte ejecutivo con resumen de supply, liquidez y quema.

Comando: powershell ./scripts/governance-executive.ps1

Variables .env: balances.csv, burn-log.json

13 Script: governance-master.ps1

Descripción: Consolida reportes multianuales y métricas globales.

Comando: powershell ./scripts/governance-master.ps1

Variables .env: balances.csv, deployments.csv

14 Script: governance-multiyear.ps1

Descripción: Reporte de gobernanza a varios años con acumulados.

Comando: powershell ./scripts/governance-multiyear.ps1

Variables .env: balances.csv, burn-log.json

15 Script: setup-doa.ps1

Descripción: Script de PowerShell para inicializar entorno DOA en Windows.

Comando: powershell ./scripts/setup-doa.ps1

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

16 Script: setup-doa.sh

Descripción: Script de Bash para inicializar entorno DOA en Linux/Mac.

Comando: bash ./scripts/setup-doa.sh

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

17 Archivo: hardhat.config.js

Descripción: Configuración de Hardhat para compilación y despliegue de contratos.

Comando: npx hardhat compile

Variables .env: NETWORK, PRIVATE_KEY_OWNER

18 Script: distribute.js

Descripción: Distribuye tokens a una lista de direcciones.

Comando: node scripts/distribute.js

Variables .env: PRIVATE_KEY_OWNER, RECIPIENTS.csv

19 Script: distributelnitial.js

Descripción: Realiza la distribución inicial del supply según configuración.

Comando: node scripts/distributelnitial.js

Variables .env: SUPPLY_CONFIG.json

20 Script: distributeSupply.js

Descripción: Distribuye tokens según porcentajes definidos en un plan.

Comando: node scripts/distributeSupply.js

Variables .env: PRIVATE_KEY_OWNER, DISTRIBUTION_PLAN.json

21 Script: distributeTokens.js

Descripción: Ejecuta el flujo completo de distribución de tokens.

Comando: node scripts/distributeTokens.js

Variables .env: Todas las necesarias para distribución (PRIVATE_KEY_OWNER, CONFIG)

22 Script: transferTokens.js

Descripción: Transfiere DOA a una wallet específica.

Comando: node scripts/transferTokens.js

Variables .env: PRIVATE_KEY_ADMIN, TO_ADDRESS, AMOUNT

23 Script: burnByPercentage.js

Descripción: Calcula y quema un porcentaje del balance de la wallet (ejemplo: 2%).

Comando: node scripts/burnByPercentage.js

Variables .env: PRIVATE_KEY_OWNER, CONTRACT_ADDRESS

24 Script: burnByPercentage.js

Descripción: Calcula y quema un porcentaje del balance de la wallet (ejemplo: 2%).

Comando: node scripts/burnByPercentage.js

Variables .env: PRIVATE_KEY_OWNER, CONTRACT_ADDRESS

25 Script: burnByTransferOwner.js

Descripción: Transfiere una cantidad fija (100,000 DOA) desde la wallet del Owner al Dead Address.

Comando: node scripts/burnByTransferOwner.js

Variables .env: PRIVATE_KEY_OWNER, CONTRACT_ADDRESS

26 Script: burnByTransfer.js

Descripción: Quema una cantidad fija (100,000 DOA) desde la cuenta Admin/Reserva hacia el Dead Address.

Comando: node scripts/burnByTransfer.js

Variables .env: PRIVATE_KEY_ADMIN, CONTRACT_ADDRESS

27 Script: logBurn.js

Descripción: Registra cada quema en burn-log.json con hash, cantidad y timestamp.

Comando: node scripts/logBurn.js

Variables .env: TX_HASH, AMOUNT, TIMESTAMP

28 Script: deploy.js

Descripción: Despliega el contrato base DOA en la red seleccionada.

Comando: node scripts/deploy.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

29 Script: deployAll.js

Descripción: Despliega múltiples contratos en lote (token, factory, router).

Comando: node scripts/deployAll.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

30 Script: deployDex.js

Descripción: Despliega el DEX o router personalizado.

Comando: node scripts/deployDex.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

31 Script: deployFactory.js

Descripción: Despliega la factory de pares/pools para el DEX.

Comando: node scripts/deployFactory.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

32 Script: deployProxy.js

Descripción: Despliega un contrato proxy (UUPS/Transparent) para actualizaciones futuras.

Comando: node scripts/deployProxy.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC

33 Script: deployTokenWithLiquidity.js

Descripción: Despliega el token DOA y añade liquidez inicial automáticamente.

Comando: node scripts/deployTokenWithLiquidity.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC, AMOUNT,
BASE_TOKEN_ADDRESS

34 Script: deployAndLiquidity.js

Descripción: Flujo combinado de despliegue del token y adición de liquidez.

Comando: node scripts/deployAndLiquidity.js

Variables .env: PRIVATE_KEY_OWNER, POLYGON_RPC, LIQUIDITY_AMOUNT

35 Script: fullFlow.js

Descripción: Flujo completo local (deploy, pares, liquidez).

Comando: node scripts/fullFlow.js

Variables .env: Todas las necesarias para deploy y liquidez

36 Script: fullFlowMainnet.js

Descripción: Flujo completo en mainnet (producción).

Comando: node scripts/fullFlowMainnet.js

Variables .env: Todas las necesarias para deploy y liquidez

37 Script: addLiquidity.js

Descripción: Añade liquidez a un par específico con cantidades definidas de token y base.

Comando: node scripts/addLiquidity.js

Variables .env: PRIVATE_KEY_ADMIN, PAIR_ADDRESS, AMOUNT

38 Script: addLiquidityAll.js

Descripción: Añade liquidez a todos los pares configurados en lote.

Comando: node scripts/addLiquidityAll.js

Variables .env: PRIVATE_KEY_ADMIN, LIQUIDITY_CONFIG

39 Script: addLiquiditySafe.js

Descripción: Añade liquidez con validaciones de slippage y límites de seguridad.

Comando: node scripts/addLiquiditySafe.js

Variables .env: PRIVATE_KEY_ADMIN, SLIPPAGE_LIMIT

40 Script: addLiquidityMainnet.js

Descripción: Añade liquidez en mainnet con parámetros de producción.

Comando: node scripts/addLiquidityMainnet.js

Variables .env: PRIVATE_KEY_ADMIN, POLYGON_RPC, AMOUNT

41 Script: addLiquidityFromPairs.js

Descripción: Itera sobre una lista de pares existentes y añade liquidez según configuración.

Comando: node scripts/addLiquidityFromPairs.js

Variables .env: PRIVATE_KEY_ADMIN, PAIRS_LIST.json

42 Script: add-liquidez.js

Descripción: Versión en español/alias funcional de addLiquidity.

Comando: node scripts/add-liquidez.js

Variables .env: PRIVATE_KEY_ADMIN, PAIR_ADDRESS, AMOUNT

43 Script: createPair.js

Descripción: Crea un nuevo par DOA/base en el DEX.

Comando: node scripts/createPair.js

Variables .env: PRIVATE_KEY_OWNER, TOKEN_ADDRESS, BASE_ADDRESS

44 Script: createPool.js

Descripción: Inicializa una pool con parámetros de fee y rango de ticks.

Comando: node scripts/createPool.js

Variables .env: PRIVATE_KEY_OWNER, FEE, TICK_RANGE

45 Script: createPairAndAddLiquidity.mjs

Descripción: Crea un par y añade liquidez inmediatamente en un único flujo.

Comando: node scripts/createPairAndAddLiquidity.mjs

Variables .env: PRIVATE_KEY_OWNER, TOKEN_ADDRESS, BASE_ADDRESS, AMOUNT

46 Script: removeLiquidity.js

Descripción: Retira liquidez de un par específico.

Comando: node scripts/removeLiquidity.js

Variables .env: PRIVATE_KEY_ADMIN, PAIR_ADDRESS

47 Script: rollbackLiquidity.js

Descripción: Revierte liquidez a un estado anterior usando registros de despliegue.

Comando: node scripts/rollbackLiquidity.js

Variables .env: PRIVATE_KEY_ADMIN, DEPLOYMENTS.json

48 Script: runLiquidity.mjs

Descripción: Flujo automatizado para añadir y gestionar liquidez.

Comando: node scripts/runLiquidity.mjs

Variables .env: Todas las necesarias para liquidez

#49 Script: check-deploy.js

Descripción: Verifica que el contrato esté correctamente desplegado y responde a llamadas.

Comando: node scripts/check-deploy.js

Variables .env: CONTRACT_ADDRESS, POLYGON_RPC

#49-1 Script: check-owner.js

Descripción: Confirma la dirección del Owner del contrato y permisos críticos.

Comando: node scripts/check-owner.js

Variables .env: CONTRACT_ADDRESS

#50 Script: check-wallet.js

Descripción: Valida que la wallet activa tenga saldo y permisos para ejecutar operaciones.

Comando: node scripts/check-wallet.js

Variables .env: WALLET_ADDRESS

#51 Script: check-gas.js

Descripción: Estima gas para operaciones y sugiere límites/fees según la red.

Comando: node scripts/check-gas.js

Variables .env: POLYGON_RPC

#52 Script: checkBalances.js

Descripción: Obtiene balances de una lista de wallets para el token DOA.

Comando: node scripts/checkBalances.js

Variables .env: WALLETS.csv, CONTRACT_ADDRESS

#53 Script: checkBalancesAdmin.js

Descripción: Revisa específicamente balances de las cuentas administrativas.

Comando: node scripts/checkBalancesAdmin.js

Variables .env: PRIVATE_KEY_ADMIN, CONTRACT_ADDRESS

#54 Script: checkBalancesOwner.js

Descripción: Consulta y reporta el balance del Owner.

Comando: node scripts/checkBalancesOwner.js

Variables .env: PRIVATE_KEY_OWNER, CONTRACT_ADDRESS

#55 Script: checkAllBalances.js

Descripción: Resume balances de múltiples wallets y tokens en un informe global.

Comando: node scripts/checkAllBalances.js

Variables .env: WALLETS.csv, CONTRACT_ADDRESS

#56 Script: checkPair.js

Descripción: Verifica existencia y estado de un par (direcciones, reservas, sync).

Comando: node scripts/checkPair.js

Variables .env: PAIR_ADDRESS

#57 Script: checkPools.js

Descripción: Revisa una pool específica y reporta métricas clave.

Comando: node scripts/checkPools.js

Variables .env: POOL_ADDRESS

#58 Script: checkPoolsAll.js

Descripción: Itera sobre todas las pools configuradas y genera un estado consolidado.

Comando: node scripts/checkPoolsAll.js

Variables .env: POOLS.json

#59 Script: checkReserves.js

Descripción: Consulta reservas del par/pool para evaluar profundidad y slippage.

Comando: node scripts/checkReserves.js

Variables .env: PAIR_ADDRESS

#60 Script: checkTotalSupply.js

Descripción: Muestra el totalSupply actual del token y detecta cambios por mint/burn.

Comando: node scripts/checkTotalSupply.js

Variables .env: CONTRACT_ADDRESS

#61 Script: analyzeEvents.js

Descripción: Analiza eventos on-chain (Transfer, Approval, Mint/Burn) para trazabilidad y auditoría.

Comando: node scripts/analyzeEvents.js

Variables .env: CONTRACT_ADDRESS, POLYGON_RPC

#62 Script: auditToken.js

Descripción: Recorre estados y configuraciones del token para generar un informe auditable (balances, supply, permisos).

Comando: node scripts/auditToken.js

Variables .env: CONTRACT_ADDRESS, PRIVATE_KEY_OWNER

#63 Script: exportDeploymentsToCSV.cjs

Descripción: Exporta registros de despliegues a un archivo CSV para análisis externo.

Comando: node scripts/exportDeploymentsToCSV.cjs

Variables .env: DEPLOYMENTS.json

#64 Script: deploymentsReport.cjs

Descripción: Genera un reporte resumido de despliegues realizados.

Comando: node scripts/deploymentsReport.cjs

Variables .env: DEPLOYMENTS.json

#65 Script: deploymentsFullReport.cjs

Descripción: Genera un reporte detallado y completo de todos los despliegues.

Comando: node scripts/deploymentsFullReport.cjs

Variables .env: DEPLOYMENTS.json

#66

Script: getLastDeployment.cjs

Descripción: Obtiene el último despliegue registrado en el sistema.

Comando: node scripts/getLastDeployment.cjs

Variables .env: DEPLOYMENTS.json

#67 Script: diagnose.js

Descripción: Ejecuta un diagnóstico general del sistema DOA (contratos, liquidez, supply).

Comando: node scripts/diagnose.js

Variables .env: Todas las necesarias para deploy y liquidez

#68 Script: testBalance.js

Descripción: Testea el balance de una wallet específica.

Comando: node scripts/testBalance.js

Variables .env: WALLET_ADDRESS, CONTRACT_ADDRESS

#69 Script: testRpc.js

Descripción: Verifica la conexión RPC de Polygon.

Comando: node scripts/testRpc.js

Variables .env: POLYGON_RPC

#70 Script: testEthRpc.js

Descripción: Verifica la conexión RPC de Ethereum.

Comando: node scripts/testEthRpc.js

Variables .env: ETH_RPC

#71 Script: testRpcCombined.js

Descripción: Testea múltiples RPC en conjunto para validar redundancia.

Comando: node scripts/testRpcCombined.js

Variables .env: RPC_LIST.json

#72 Script: testSwaps.cjs

Descripción: Prueba swaps en un entorno controlado para validar lógica de intercambio.

Comando: node scripts/testSwaps.cjs

Variables .env: ROUTER_ADDRESS, AMOUNT

#73 Script: quickCheck.js

Descripción: Realiza una validación rápida de despliegue y estado del sistema.

Comando: node scripts/quickCheck.js

Variables .env: DEPLOYMENTS.json

#74 Script: getPairAddress.js

Descripción: Obtiene la dirección de un par DOA/base en el DEX.

Comando: node scripts/getPairAddress.js

Variables .env: TOKEN_ADDRESS, BASE_ADDRESS

#75 Script: check-nonce.js

Descripción: Verifica el nonce actual de una wallet para prevenir errores de transacción.

Comando: node scripts/check-nonce.js

Variables .env: WALLET_ADDRESS

#76 Script: cleanEntityTags.js

Descripción: Limpia etiquetas o metadata asociada a direcciones y entidades del sistema para mantener orden y consistencia.

Comando: node scripts/cleanEntityTags.js

Variables .env: ENTITY_LIST.json

#77 Archivo: .gitignore

Descripción: Define qué archivos y carpetas se excluyen del control de versiones (ejemplo: .env, logs, build).

Uso: Se mantiene en la raíz del proyecto para proteger datos sensibles y evitar subir artefactos innecesarios.

#78 Carpeta: /contracts

Descripción: Contiene los contratos inteligentes principales (DOAToken.sol, Proxy, Factory, Router).

Uso: Aquí se define la lógica del token, funciones de burn, distribución y liquidez.

#79 Carpeta: /scripts

Descripción: Conjunto de scripts operativos para despliegue, liquidez, quema, distribución, transferencias, validaciones y auditoría.

Uso: Cada archivo ejecuta una acción específica sobre el contrato o el ecosistema DOA.

#80 Carpeta: /logs

Descripción: Almacena registros auditables de operaciones (liquidity.log, burn-log.json, liquidity-events.log).

Uso: Fuente de trazabilidad para auditorías y dashboards.

#81 Carpeta: /docs

Descripción: Documentación técnica, manuales de uso, README y paquetes de cumplimiento (AML/KYC).

Uso: Se utiliza para onboarding de colaboradores y listados en exchanges.

#82 Carpeta: /dashboard

Descripción: Archivos del panel visual (App.js, balances.js, index.html, iconos).

Uso: Interfaz gráfica para monitorear balances, liquidez y ejecutar funciones DOA.

#83 Carpeta: /config

Descripción: Archivos de configuración (hardhat.config.js, .env.example).

Uso: Define parámetros de red, claves y settings de despliegue.

#84 Carpeta: /tests

Descripción: Scripts de prueba y validación (testBalance.js, testRpc.js, testSwaps.cjs).

Uso: Garantiza que las funciones del contrato y scripts operativos funcionen correctamente.

#85

Archivo: .env

Descripción: Archivo principal de variables de entorno (claves privadas, RPC, configuraciones de red).

Uso: Fuente crítica para todos los scripts y despliegues.

#86

Archivo: .last_deploy

Descripción: Registro del último despliegue realizado (hash, fecha, contrato).

Uso: Referencia rápida para auditoría y rollback.

#87

Archivo: addresses.json

Descripción: Contiene direcciones de contratos y wallets clave (Owner, Admin, Pools).

Uso: Fuente para scripts de validación y transferencias.

#88

Archivo: balances.csv

Descripción: Exportación de balances de wallets en formato CSV.

Uso: Fuente para reportes de gobernanza y auditoría.

#89

Archivo: burn-log.json

Descripción: Registro auditable de todas las quemas de tokens.

Uso: Se actualiza automáticamente desde scripts de burn.

#90

Archivo: deployments.json / deployments.csv / deployments.old

Descripción: Historial de despliegues de contratos (actual, exportado y versiones previas).

Uso: Fuente para validaciones, reportes y rollback.

#91

Archivo: lp-distribution.json

Descripción: Configuración y registro de distribución de LP tokens.

Uso: Fuente para scripts de distribución y auditoría de liquidez.

#92

Archivo: pairs.json

Descripción: Configuración de pares DOA/base en el DEX.

Uso: Fuente para scripts de createPair y addLiquidity.

#93

Archivo: ownership-signature.txt

Descripción: Firma criptográfica que valida la propiedad del contrato por el Owner.

Uso: Evidencia para auditorías y listados en CEX.

#94

Archivo: README.md

Descripción: Documentación base del proyecto DOATokenV2.

Uso: Guía inicial para colaboradores y exchanges.

#95

Archivo: SECURITY.md

Descripción: Políticas de seguridad, buenas prácticas y manejo de claves.

Uso: Cumplimiento y auditoría de seguridad.

#96

Carpeta: /audit

Descripción: Contiene reportes de auditorías externas de contratos y liquidez.

Uso: Evidencia para cumplimiento y listados en CEX.

#97

Carpeta: /automation

Descripción: Scripts automatizados para tareas recurrentes (deploy, liquidez, validaciones).

Uso: Facilita operaciones sin intervención manual.

#98

Carpeta: /compliance

Descripción: Documentación AML/KYC y certificación legal de que el token no es valor.

Uso: Paquete de cumplimiento para exchanges y reguladores.

#99

Carpeta: /cex-application

Descripción: Archivos de aplicación oficial para listados en CEX.

Uso: Formularios, métricas y documentación consolidada.

#100

Carpeta: /FundadorUnico

Descripción: Documentación adaptada al perfil de fundador único.

Uso: Evidencia organizacional para exchanges y auditores.

#101

Carpeta: /reports

Descripción: Reportes adicionales de métricas, balances y liquidez.

Uso: Complemento para gobernanza y auditoría.

#102

Carpeta: /signer

Descripción: Archivos relacionados con firmas digitales y claves.

Uso: Validación de transacciones y despliegues.

#103

Carpeta: /tokenlist y /tokenlists

Descripción: Listas oficiales de tokens en formato JSON.

Uso: Registro en DEX y validación de pares.

#104

Carpeta: /data

Descripción: Datasets auxiliares para scripts y reportes.

Uso: Fuente de información adicional para análisis.

#105

Carpeta: /docs.old

Descripción: Documentación previa y versiones antiguas.

Uso: Referencia histórica y respaldo.

#106

Archivo: BCO.bdcfa9d8-d70e-4b42-8db7-05a523dfae8b.png

Descripción: Logo/branding del proyecto DOA.

Uso: Identidad visual para dashboard y presentaciones.

#107

Archivo: liquidityAccumulatedChart.png

Descripción: Gráfico acumulado de liquidez.

Uso: Visualización en reportes y dashboards.

#108

Archivo: liquidityChart.png

Descripción: Gráfico de liquidez en tiempo real.

Uso: Monitoreo y reportes de gobernanza.

#109 – Staking Module

- Permite a los usuarios bloquear DOA para obtener recompensas.
- Parámetros configurables: periodos de bloqueo, tasas variables, penalizaciones.

- Logs auditables en [staking.json](#).

#110 – Governance Module

- Uso del token DOA como poder de voto.
- Integración con propuestas comunitarias.
- Ejecución automática de decisiones aprobadas.

#111 – Treasury Module

- Contrato de tesorería que recibe comisiones de swaps o transferencias.
- Fondos destinados a desarrollo, marketing o recompensas.
- Reportes en [treasury.json](#).

#112 – Liquidity Incentives Module

- Distribución automática de recompensas a proveedores de liquidez.
- Integración con [deployments.json](#) para trazabilidad.
- Métricas en dashboard.

#113 – Proxy Upgradability

- Patrón de proxy para actualizar lógica sin cambiar dirección.
- Mantiene compatibilidad con exchanges y dashboards.

#114 – Cross-chain Bridge Adapter

- Extiende compatibilidad con protocolos de puente.
- Permite circulación de DOA en múltiples cadenas.
- Requiere pruebas de seguridad adicionales.

#115 – Fee Mechanisms

- Tasas dinámicas en transferencias.
- Redirección de porcentajes a tesorería o quema de tokens.
- Control de inflación y generación de valor.

#116 – Compliance Controls

- Funciones de pausa y blacklist.

- Cumplimiento de requisitos regulatorios.
- Documentación en [compliance.json](#).
- ☒ Scripts de validación, gobernanza, setup, distribución, transferencias, burn, deploy, liquidez, chequeo, auditoría, diagnóstico, utilidades.
- ☒ Archivos raíz críticos (.env, balances.csv, burn-log.json, deployments.json, lp-distribution.json, pairs.json, ownership-signature.txt, README.md, SECURITY.md).
- ☒ Carpetas auxiliares (/audit, /automation, /compliance, /cex-application, /FundadorUnico, /reports, /signer, /tokenlist, /data, /docs.old).
- ☒ Assets gráficos (logo, liquidity charts).
- ☒ Índice general de carpetas (#78–#84).

En otras palabras: tu **manual técnico DOATokenV2** ya está completo, modular y listo para copiar/pegar en Word o mantener como README auditable en el repo.